VERITAS UNIVERSITY, ABUJA · DEPARTMENT OF SOFTWARE ENGINEERING

VUA-SEN307 Database Systems — Week 5 Practical

Lab 3: Data Definition Language (DDL): Creating Schemas

Duration: 3 hours **Tools:** MySQL Server, MySQL Workbench, Git **Platforms:** Windows & macOS

Builds on: your Week 4 ER diagram and 3NF schema (`week04/lab2_model.mwb`)

Objectives

By the end of today you should be able to:

- turn your Week 4 ER diagram into physical tables using `CREATE TABLE`
- choose suitable data types (INT, VARCHAR, DATE, DECIMAL, ENUM, TEXT)
- apply constraints: PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK and DEFAULT
- change a table after it is created with `ALTER TABLE`

Session plan

Time	Activity
0:00 – 0:15	Recap: open your Week 4 diagram and compare it with the class reference schema
0:15 – 0:40	**Part 1:** Data types and constraints
0:40 – 1:30	**Part 2:** Create the six tables
1:30 – 1:40	Break
1:40 – 2:10	**Part 3:** ALTER TABLE and testing the constraints
2:10 – 2:40	**Part 4:** Verify the schema and reverse-engineer the diagram
2:40 – 3:00	GitHub commit and submission check

Recap (15 min)

1. Open MySQL Workbench and click the **Level 1 Connection** tile.
2. Open your Week 4 model: **File → Open Model →** `week04/lab2_model.mwb`.
3. Compare your tables with the class reference schema in Part 2. Note any differences in your notes. **Everyone uses the reference schema from today**, so the code in later labs works for the whole class.

Part 1: Data Types and Constraints (25 min)

Data type	Use it for	Example in our schema
INT	Whole numbers, especially IDs	`student_id`
TINYINT	Small whole numbers (−128 to 127)	`credit_units`
VARCHAR(n)	Text up to *n* characters	`full_name VARCHAR(100)`
TEXT	Long text	`description`
DATE	Calendar dates (YYYY-MM-DD)	`date_of_birth`
DECIMAL(p,s)	Exact numbers such as money or GPA	*(not used yet)*
ENUM(...)	One value from a fixed list	`gender ENUM('M','F')`

Constraint	What it guarantees
PRIMARY KEY	Every row has a unique, non-empty identifier
FOREIGN KEY	A value must exist in the parent table (referential integrity)
NOT NULL	The column can't be left empty
UNIQUE	No two rows can share the value
CHECK	The value must satisfy a condition
DEFAULT	The value to use when none is given
AUTO_INCREMENT	MySQL numbers new rows automatically

Part 2: Create the Six Tables (50 min)

1. In Workbench, open a new SQL tab: **File → New Query Tab**. On Windows the shortcut is **Ctrl + T**; on macOS it is **Cmd + T**.
2. Select your database (replace `20231234` with your matric number):

   ```
   USE sen307_20231234;
   ```

3. Type the script below. Create the tables **in this order**: a parent table must exist before a child table can reference it.

```
CREATE TABLE Department (
  dept_id    INT PRIMARY KEY AUTO_INCREMENT,
  dept_name  VARCHAR(100) NOT NULL UNIQUE
);

CREATE TABLE Lecturer (
  lecturer_id INT PRIMARY KEY AUTO_INCREMENT,
  staff_no    VARCHAR(20)  NOT NULL UNIQUE,
  full_name   VARCHAR(100) NOT NULL,
  email       VARCHAR(100) UNIQUE,
  dept_id     INT NOT NULL,
  FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

CREATE TABLE Student (
  student_id    INT PRIMARY KEY AUTO_INCREMENT,
  matric_no     VARCHAR(20)  NOT NULL UNIQUE,
  full_name     VARCHAR(100) NOT NULL,
  gender        ENUM('M','F'),
  date_of_birth DATE,
  dept_id       INT,
  FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

CREATE TABLE Course (
  course_id    INT PRIMARY KEY AUTO_INCREMENT,
  course_code  VARCHAR(10)  NOT NULL UNIQUE,
  course_title VARCHAR(150) NOT NULL,
  credit_units TINYINT NOT NULL DEFAULT 3,
  description  TEXT,
  dept_id      INT,
  FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);
```

```
CREATE TABLE Enrolment (
  enrolment_id     INT PRIMARY KEY AUTO_INCREMENT,
  student_id       INT NOT NULL,
  course_id        INT NOT NULL,
  academic_session VARCHAR(9) NOT NULL DEFAULT '2026/2027',
  semester         ENUM('First','Second') NOT NULL DEFAULT 'First',
  enrolled_on      DATE DEFAULT (CURRENT_DATE),
  UNIQUE (student_id, course_id, academic_session),
  FOREIGN KEY (student_id) REFERENCES Student(student_id),
  FOREIGN KEY (course_id)  REFERENCES Course(course_id)
);

CREATE TABLE CourseAllocation (
  allocation_id    INT PRIMARY KEY AUTO_INCREMENT,
  course_id        INT NOT NULL,
  lecturer_id      INT NOT NULL,
  academic_session VARCHAR(9) NOT NULL DEFAULT '2026/2027',
  UNIQUE (course_id, lecturer_id, academic_session),
  FOREIGN KEY (course_id)   REFERENCES Course(course_id),
  FOREIGN KEY (lecturer_id) REFERENCES Lecturer(lecturer_id)
);

ALTER TABLE Student ADD COLUMN phone VARCHAR(15) AFTER full_name;
ALTER TABLE Course  ADD CONSTRAINT chk_credit_units CHECK (credit_units BETWEEN 1 AND 6);
```

The last two lines are the **ALTER TABLE** step. They add a `phone` column and a `CHECK` constraint after the tables already exist.

4. Run the script. On Windows, press **Ctrl + Shift + Enter**; on macOS, press **Cmd + Shift + Enter**. You can also click the lightning-bolt icon.
5. Save the script as `schema.sql`: **File → Save Script As**.

Why six tables, not five? In Week 4 you found that Course–Lecturer is a many-to-many relationship across sessions. `CourseAllocation` is the junction table that stores it. `Enrolment` is the junction table for Student–Course.

Part 3: Testing the Constraints (30 min)

Each statement below **breaks a rule on purpose**. Run them one at a time and copy each error message into your notes. Every one should fail, so no rows are saved.

```
-- Test 1: NOT NULL (should fail: dept_name is required)
INSERT INTO Department (dept_name) VALUES (NULL);

-- Test 2: CHECK (should fail: credit_units must be 1 to 6)
INSERT INTO Course (course_code, course_title, credit_units) VALUES ('TST101', 'Test', 9);

-- Test 3: FOREIGN KEY (should fail: department 999 does not exist)
INSERT INTO Student (matric_no, full_name, dept_id) VALUES ('TEST001', 'Test', 999);

-- Test 4: ENUM (should fail: gender must be 'M' or 'F')
INSERT INTO Student (matric_no, full_name, gender) VALUES ('TEST002', 'Test', 'X');

-- Confirm every table is still empty
SELECT COUNT(*) FROM Department;
SELECT COUNT(*) FROM Course;
SELECT COUNT(*) FROM Student;
```

Your turn: Write one more statement that breaks the **UNIQUE** constraint on `Department.dept_name`. Hint: you will need to insert the same name twice. Delete the row afterwards with `DELETE FROM Department WHERE dept_name = '...';`

Part 4: Verify the Schema (30 min)

1. Run these and take a screenshot of each `DESCRIBE` result:

```
SHOW TABLES;
DESCRIBE Department;
DESCRIBE Lecturer;
DESCRIBE Student;
DESCRIBE Course;
DESCRIBE Enrolment;
DESCRIBE CourseAllocation;
SHOW CREATE TABLE Enrolment;
```

2. **Reverse-engineer the diagram:** go to **Database → Reverse Engineer**, choose the **Level 1 Connection**, tick your `sen307_<matric_number>` database and click through to **Finish**. Workbench draws an EER diagram from your real tables.
3. Compare it with your Week 4 diagram. Write two or three sentences on what differs and why.
4. Export it with **File → Export → Export as PNG** and save it as `lab3_erd.png`.

Push to GitHub (same on both)

1. Create this week's folder in your repository:

```
cd sen307-labs
mkdir week05
```

2. Copy `schema.sql`, your `DESCRIBE` screenshots and `lab3_erd.png` into `week05/`. On Windows, use File Explorer; on macOS, use Finder.
3. Commit and push:

```
git add .
git commit -m "Week 5: DDL schema with constraints"
git push
```

Submission Checklist (before you leave)

- ☐ `week05/schema.sql` containing all CREATE TABLE and ALTER TABLE statements
- ☐ Screenshots of the `DESCRIBE` output for each of the six tables
- ☐ The constraint-test error messages in your notes, plus your own UNIQUE test
- ☐ `week05/lab3_erd.png` (reverse-engineered diagram)
- ☐ All files pushed to GitHub

Next week (Week 6): you will fill these tables with data. Don't drop or rename any tables or columns.